

Halbach Array MRI: Modifications on Ring Parameters & Analyzing Parameter Combinations

Hannah Jensen, 2076653, Undergraduate, Department of Computer Science
Caroline Jensen, 2278649, Undergraduate, Department of Computer Science

Problem Statement:

Introduction:

Halbach arrays are specialized arrangements of magnets that enhance the magnetic field strength on one side while minimizing it on the other. In particular, this property makes them attractive for applications requiring compact, lightweight, or specifically-targeted magnetic field configurations, including designing more portable MRI systems [1]. We plan to test what happens when you alter the gap between the permanent magnet arrays as well as what would happen by increasing the radius of the rings. These variations allow us to explore design trade-offs relevant to improving the performance, portability, and accessibility of Halbach-based MRI systems.

Objectives:

The primary goal of this project was to evaluate the magnetic field of a Halbach array MRI by systematically analyzing the impact of its geometric parameters on field strength and homogeneity. Our specific aims were:

Aim 1: To determine the effect of varying the number of rings.

Aim 2: To determine the effect of varying the ring radii.

Aim 3: To determine the effect of varying the ring separation.

Aim 4: To simulate and assess optimal parameter combinations to identify configurations that best balance high field strength with low homogeneity.

Methods:

Overview:

Given that we plan to test what happens when you alter the number of rings, ring radii sizes, and the ring separation between the permanent magnet arrays, we would need to alter field projections along the Z-axis and test the regular magnet arrays as well as the ones with the increased parameters. We will use the Python programming language in Visual Studio Code to achieve these results with the following libraries:

- **numpy & pandas:** for efficient multi-dimensional mathematical operations
- **matplotlib:** for visualization
- **csv & pickle:** for saving and loading in results

These files will include the analysis of the resulting magnetic field strength, measured with mean (mT), and the spatial variation, measured with homogeneity (ppm and mT). There is often a trade-off with these two values; the goal we have is to achieve high field strength and low variation. We will present our results through generating plots of the different resulting metrics in order to show the effects that take place with changes in the ring's spacing and radius.

Implementation:

The script used was derived from Tom O'Reilly's original *homogeneityOptimization.py* to optimize field homogeneity of Halbach arrays, and had been further modified by Robert Griffin to isolate forward field calculations [2]. We used it to test each Aim's effects on field strength and homogeneity through the use of these functions, as well as our own. For each individual aim, two of the three primary parameters were held constant while the third was varied:

- **For Aim 1 (Number of Rings):** The inner ring radii were held constant, and the ring separation was fixed at 20 mm.
- **For Aim 2 (Radii Sizes):** The number of rings was held constant at 4, and the ring separation was fixed at 20 mm.
- **For Aim 3 (Gap Sizes):** The number of rings was held constant at 4, and the inner ring radii were fixed at [148 mm, 151 mm].

The key performance metrics were calculated from the simulated magnetic field within the cylindrical field of view after the Halbach simulation is performed.

- **Mean (mT):** The average magnetic field strength within the region of interest (ROI), calculated:
 $mean(field) * 10^3$
- **Homogeneity (ppm):** The magnetic field variation, normalized by the mean field strength, calculated:
 $\frac{max(field) - min(field)}{mean(field)} * 10^6$

The following functions were defined and executed to implement the analysis workflow for each Aim:

Computational Functions:

1. `halbach_array_simulation(numRings, innerRingRadii, ..., graph)`
This core function executes only a single simulation.
 - Defines the magnet array geometry based on input parameters.
 - Creates a cylindrical mask to define the field of view.
 - Applies the cylindrical mask to the simulated field and calculates the mean and homogeneity.
 - Calls `halbachFields.createHalbach()` function to calculate the magnetic field contribution from each ring.
 - Calls `graph_fields(coordinateAxis, maskedField)` to visualize the field.
2. `run_parameter_sweep(halbach_simulation_func, ..., save_fields)`
This function automates batch processing.
 - Takes a list of parameter configurations (param_configs).
 - Iteratively calls `halbach_array_simulation` for each configuration.
 - Stores the results (parameters, field strength, homogeneity) for each run.
 - Saves and loads results using pickle and csv to avoid redundant simulations.
3. `save_results(results_list, field_data_list, filename_base)` & `load_results(filename_base)`
Saves and loads in results to help reduce runtime on multiple identical executions.

Visualization Functions:

4. `plot_field_metrics_histogram(all_results)`
Visualizes the final field strength and homogeneity values for all tested configurations in a given set in a histogram.

5. `param_homogeneity_trendline`(all_results, ..., param_extractor) & `param_field_strength_trendline`(all_results, ..., param_extractor)
Generates scatter plots with linear or quadratic regression lines to visualize the relationship between a single parameter and the resulting homogeneity or field strength.
6. `plot_parameter_pairs`(all_results)
Generates a grid of scatter plots that visualize how different Halbach array design parameters relate to one another; in particular the radius, number of rings, and gap variables. A pairwise comparison matrix is created, consisting of the three different pairs of values adjusted, with one parameter on the x-axis and another on the y-axis.
7. `plot_field_strength_vs_homogeneity`(all_results)
Plots the relationship between a Halbach array's magnetic field strength and its homogeneity for each of our simulated parameter configurations, illustrating which combination resulted in the best field strength and homogeneity values. They are plotted as individual points on a two-dimensional scatter plot.

The parameters were swept through the following ranges to systematically test their effects:

- **Number of Rings:** Tested from 2 to 20, increasing in steps of 2.
- **Ring Separation (Gap):** Tested from 12 mm to 25 mm, increasing in steps of 1 mm.
- **Inner Ring Radii:** Tested with pairs [140 mm, 143 mm] with a mean radius of 141.5 mm to pairs [156 mm, 159 mm] with a mean radius of 157.5 mm.

Results:

Aim 1: Changing the Number of Rings

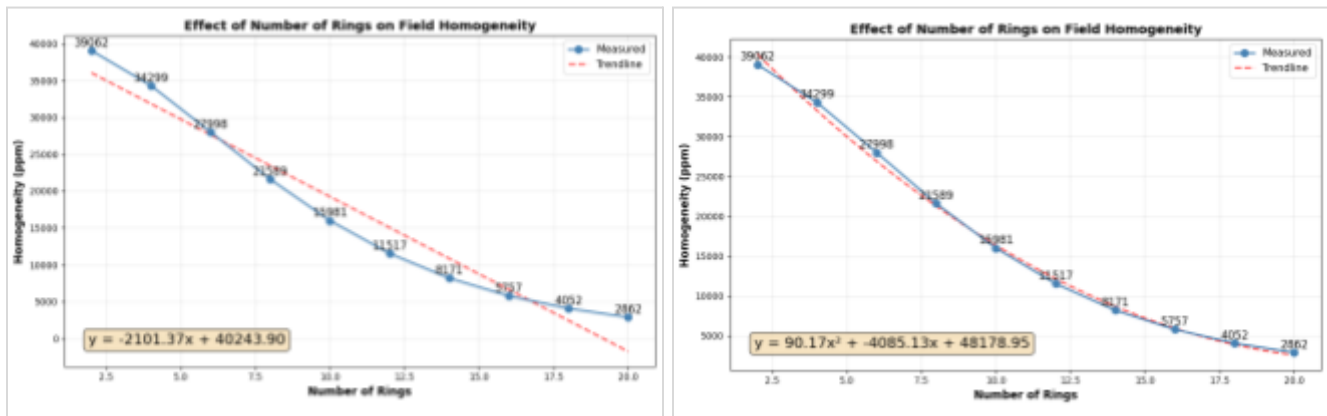


Figure 1 & 2: Effect of increasing number of rings on homogeneity (ppm), illustrated with a linear regression fit (left panel) and a quadratic regression fit (right panel).

R^2 values for the linear and quadratic regression lines are 0.9519 and 0.9968 respectively.

As shown in the figures, as the number of rings in the cylindrical Halbach array increases, the simulated homogeneity value (in ppm) decreases. This indicates that there is a significant improvement in the spatial uniformity of the internal magnetic field, which appears to slow in rate of descent as we increase the number of rings. This helps explain why the quadratic regression line is a better fit than the linear one, as indicated by the corresponding R^2 values, since the improvement shows the diminishing returns that adding each ring has, rather than following a constant slope of improvement. In an ideal, infinitely long Halbach cylinder, the magnetic field would have perfect uniformity and therefore a homogeneity value of zero. This helps explain why, physically, the improvement we see is expected. Since adding more rings better approximates the magnetization distribution of the infinite/ideal Halbach cylinder, by adding rings

we are progressively reducing the residual spatial variability in the magnetic field. Each additional ring brings the field profile closer to that perfectly uniform limit mentioned before, though with diminishing incremental benefit, further explained by figure 2's curvature above.

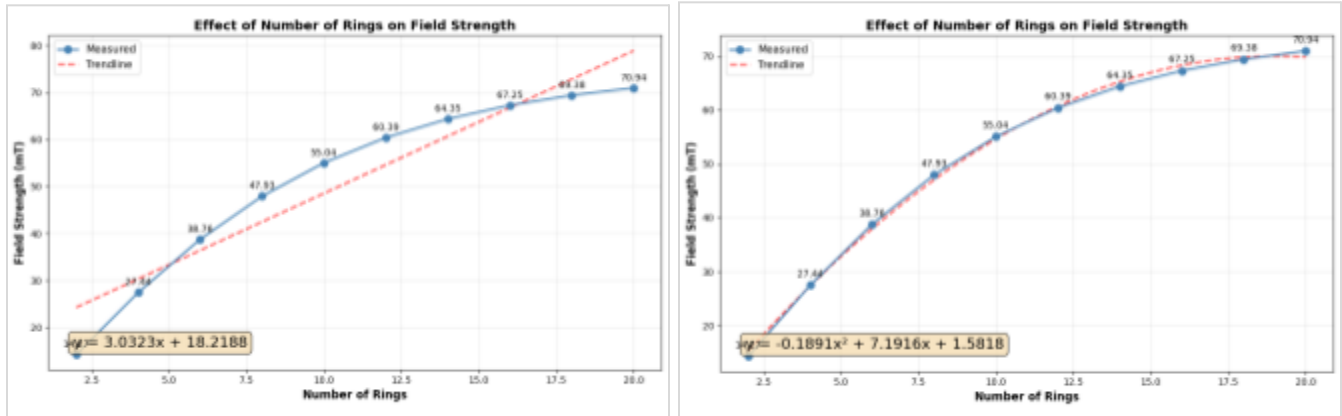


Figure 3 & 4: Effect of increasing number of rings on mean (mT), illustrated with a linear regression fit (left panel) and a quadratic regression fit (right panel).

R^2 values for the linear and quadratic regression lines are 0.9079 and 0.9982 respectively.

The figures above illustrate similar trends as the homogeneity plots shown in figures 1 & 2. We see that, as the number of rings in the cylindrical Halbach array increases, the simulated field strength also increases. Similar to the analysis above, this also reflects how extending the array more closely approximates the ideal/infinite Halbach cylinder whose continuous magnetization distribution produces a stronger internal field. The rate of increase slows as more rings are added, indicating that each additional ring contributes progressively less to the total field strength. This behavior explains why a nonlinear (quadratic) regression line provides a better fit than a linear one, as indicated by the corresponding R^2 values. In the limit of an infinitely long Halbach cylinder, the internal field reaches its theoretical maximum. Thus, as we add rings, we are steadily pushing the system toward this limit, increasing the field strength while gradually exhausting the contribution each added ring can make. There is a physical limitation with the addition of these rings, as the MRI must be not only physically feasible, but also practical. Increasing the number of rings increases the overall size and cost, complicating the construction and anatomical fit for the patient.

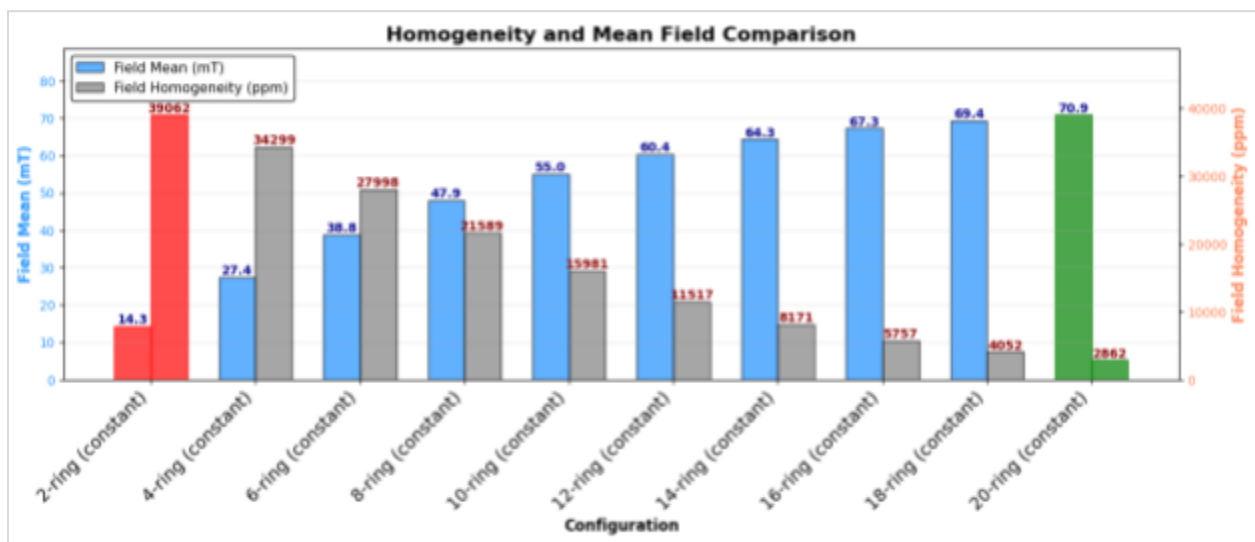


Figure 5: Comparison of final field means and homogeneities between different numbers of rings.

The figure above illustrates the clear correlation between the number of rings in the Halbach array and both key performance metrics, those being the field strength and homogeneity. When visualized together, the correlation that these two results have as the number of rings increases becomes clear, as the increase not only boosts the field strength but also enhances the spatial uniformity at a similar rate. This associated improvement reflects the conclusions we reached in the explanations above: extending the array pushes the simulation closer to our ideal, but with diminishing returns as the rings continue increasing in number.

Aim 2: Changing the Radius

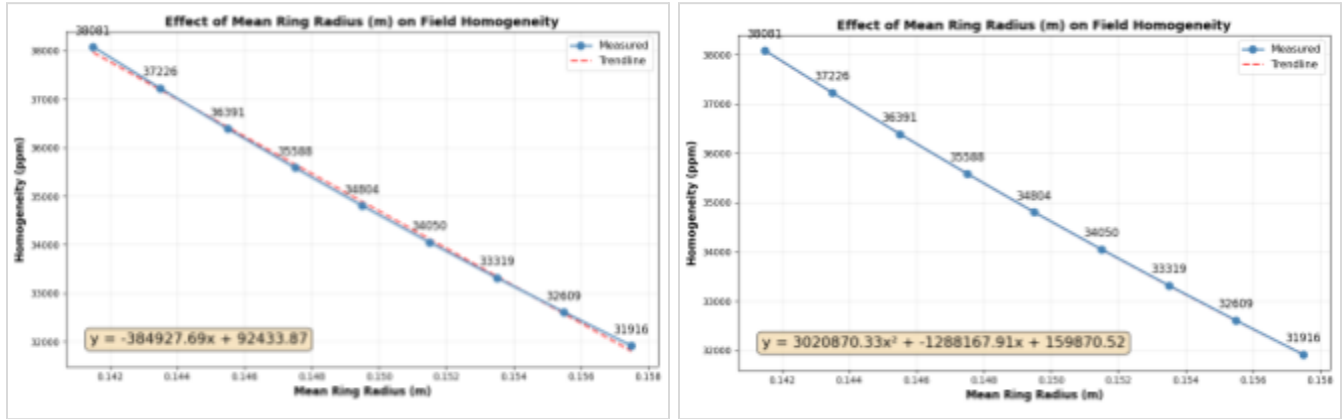


Figure 6 & 7: Effect of increasing mean ring radius on homogeneity (ppm), illustrated with a linear regression fit (left panel) and a quadratic regression fit (right panel).

The corresponding R^2 values for the linear and quadratic fits are 0.9987 and 1.000, respectively.

As shown in the figures, increasing the mean ring radius leads to a decrease in the homogeneity value. This indicates that the internal magnetic field becomes more uniform as the mean ring radius increases, improving the overall uniformity of the field. Physically, this can be explained by the fact that a larger mean ring radius reduces the influence of edge effects at the boundaries of the cylinder. As the rings become larger, the field inside the cylinder better approximates that of an ideal Halbach configuration, where the magnetic field is more evenly distributed across the internal volume, resulting in a lower homogeneity value. Since the quadratic regression line is such a near-perfect fit, this again reflects diminishing returns for increasing the ring radius. The fact that the linear regression line also fits closely suggests that the radius values tested have not yet reached the point where the homogeneity improvement begins to “bottom out,” meaning the field uniformity has not yet approached its theoretical maximum radii limit.

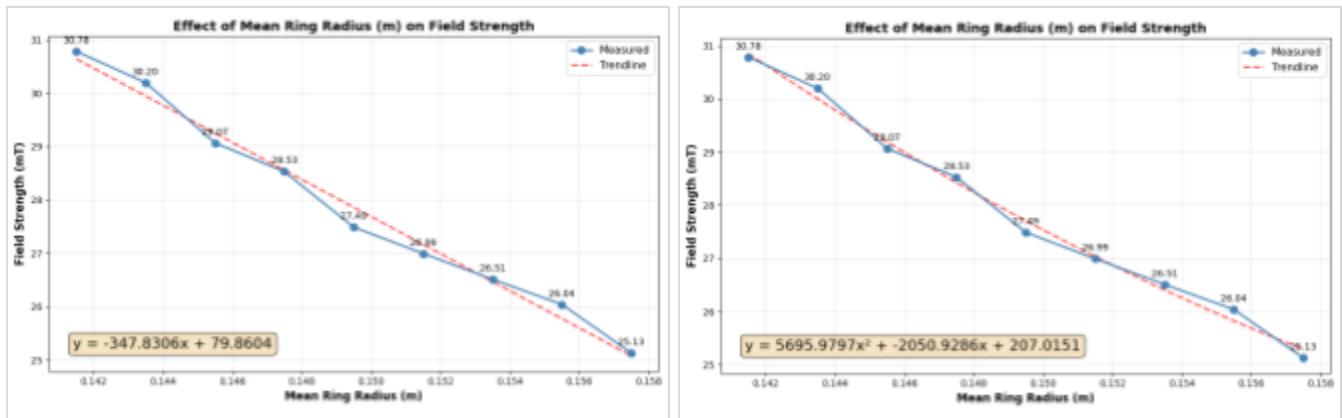


Figure 8 & 9: Effect of increasing mean ring radius on mean (mT), illustrated with a linear regression fit (left panel) and a quadratic regression fit (right panel).

R^2 values for the linear and quadratic regression lines are 0.9877 and 0.9932 respectively.

As shown in the figures, the mean magnetic field strength decreases as the ring radii increases. This inverse relationship indicates that increasing the radius of the rings reduces the overall magnetic field strength inside the simulated cylinder. This is because increasing the ring radius spreads the magnetic material over a larger volume, increasing the distance between the magnet source and the central region, therefore weakening the magnetic field. The scatterplot exhibits some small but notable spikes, which occur because the plot uses the mean radius of each ring radii pair to represent the magnetic field strength output. Since each ring has a finite thickness, the actual field strength varies across its radial extent. The strong R^2 values for both linear and quadratic fits indicate that the overall trend of decreasing field strength with increasing ring radius is well captured by these models. The slightly better fit of the quadratic model suggests some nonlinearity.

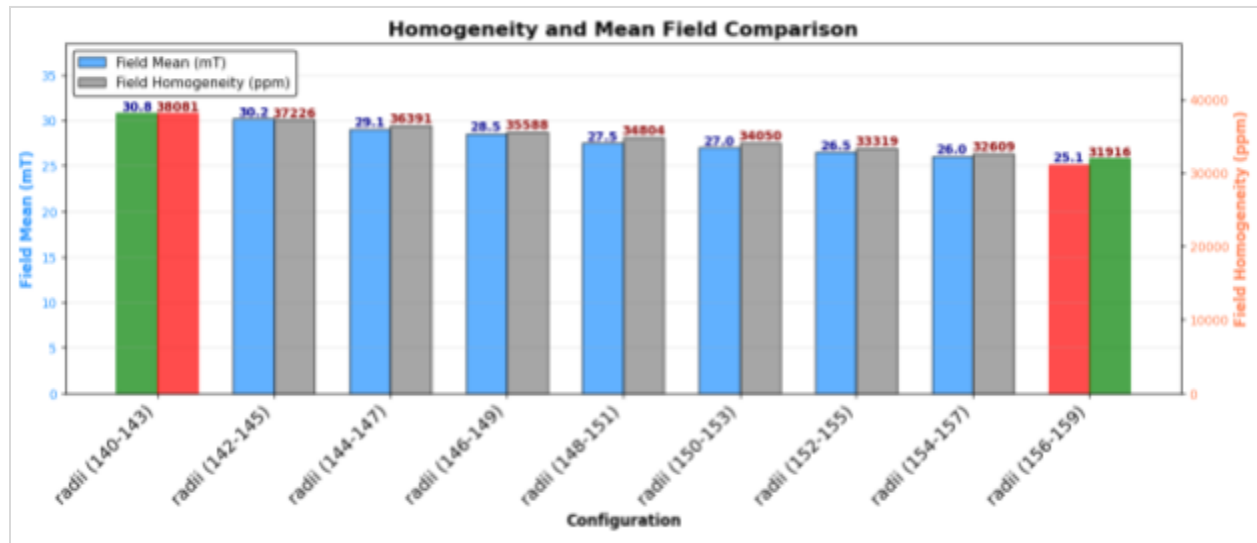


Figure 10: Comparison of final field means and homogeneities between various ranges of radii.

The results reveal a consistent trend: increasing the radius leads to improved field homogeneity, with the smallest radii [140 mm, 143 mm] showing the highest variability and one of the largest ranges [156 mm, 159 mm] producing the most uniform field. However, this improvement comes with a notable trade-off. As the radius increases, the mean field strength tends to decrease slightly, indicating that designers must balance the desire for better homogeneity with the need to maintain sufficient field magnitude. This highlights a key design consideration for Halbach-based MRI systems. Larger radii can stabilize the field, but it may require other methods to help preserve an acceptable field strength.

Aim 3: Changing the Ring Separation

For testing the ring separation, we held the number of rings constant at 4 mm and the inner ring radii constant at [148 mm, 152 mm]. Our ring separation values ranged from 12 to 25 with a step-size of 1. Physically, 20 mm (2cm) is the highest appropriate ring separation, as going any higher than this will result in each ring functionally acting independently of one another, and the space required for the Halbach array becomes increasingly infeasible for implementation.

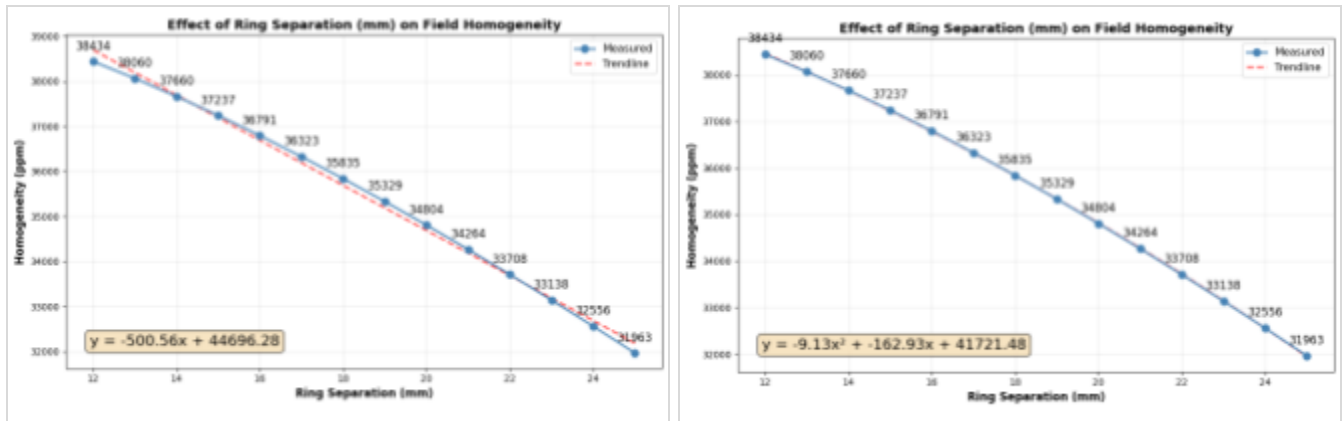


Figure 11 & 12: Effect of increasing ring separation on homogeneity (ppm), illustrated with a linear regression fit (left panel) and a quadratic regression fit (right panel).

R^2 values for the linear and quadratic regression lines are 0.9957 and 1.000 respectively.

As shown by the figures above, as the ring separation increases, the homogeneity decreases. This means that the image will have an increasingly uniform magnetic field as the ring separation increases. This is explained due to the fact that the wider spacing reduces interference between each ring's individual magnetic fields to their adjacent rings. The result is a smoother, more spatially uniform magnetic field, and as the rings get further apart, each ring's contribution behaves more like an independent field source rather than part of one total magnetic structure. This reduction minimizes the local distortions between each ring, causing the combined field to calculate as a more uniform overall distribution.

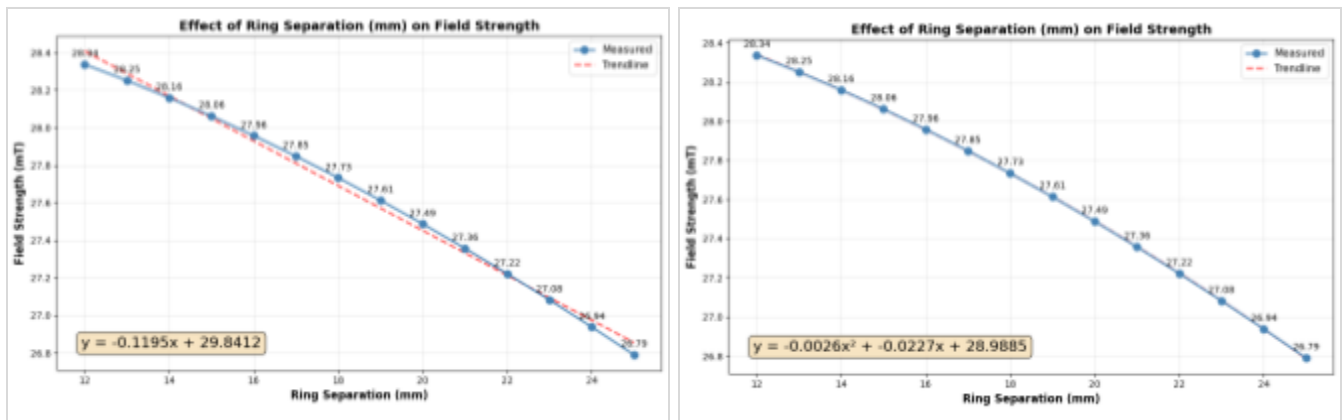


Figure 13 & 14: Effect of increasing ring separation on mean (mT), illustrated with a linear regression fit (left panel) and a quadratic regression fit (right panel).

R^2 values for the linear and quadratic regression lines are 0.9939 and 1.000 respectively.

As the ring separation increases, we can see from the figures above that the mean field strength decreases. This happens because increasing the distance between the rings weakens the additive overlapping of their magnetic fields. When the rings are close together, their fields overlap strongly and reinforce one another, producing a higher overall magnetic field as desired. However, as the spacing grows, each ring sits farther from the imaging region, and its contribution diminishes with distance, which effectively compromises the goal of maintaining one strong, unified field. Essentially, the increasing gap size reduces the coupling effects between rings, meaning their fields no longer combine as efficiently. As a result, the total magnetic field becomes weaker the further apart the rings are placed.

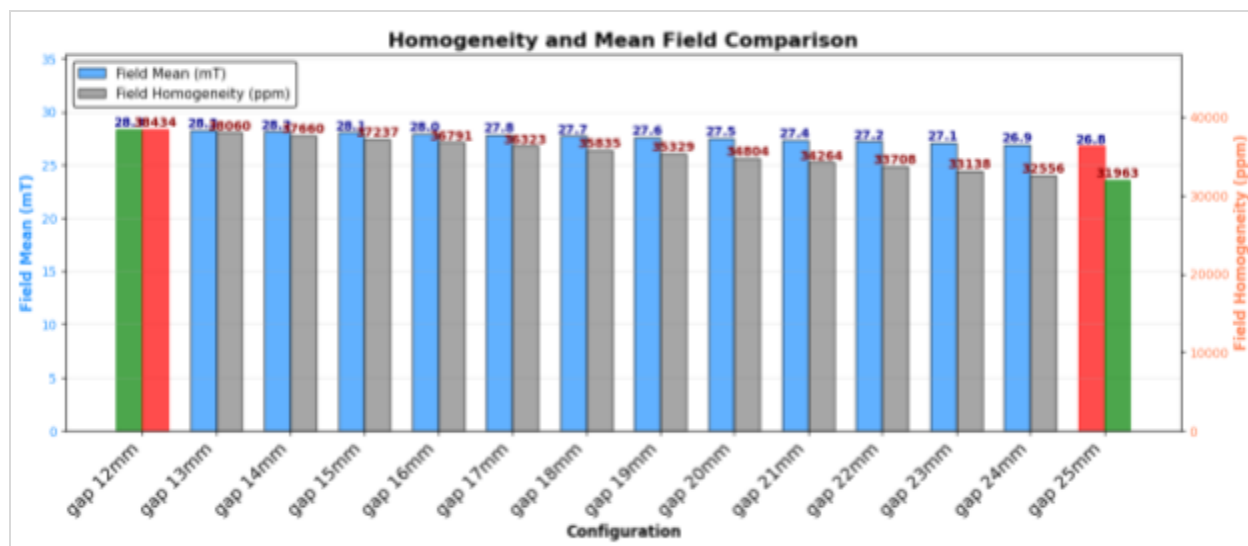


Figure 15: Comparison of final field means and homogeneities between different ring separation sizes.

In this figure, we can see another clear trade-off when adjusting the ring separation size. As the ring separation increases, the homogeneity improves, indicating that the magnetic field becomes more uniform. This is occurring due to the wider spacing reducing interference, expanded on in the analysis above. At the same time, however, the mean field strength is also steadily decreasing with the increasing ring separation. The reduced additivity between rings also means their fields no longer reinforce one another as effectively, resulting in a lower net magnetic field.

Aim 4: Optimal Parameter Combinations

In this section, the aim was to find a combination of parameters that resulted in the best overall field, balancing high field strength with low homogeneity. Aims 1 through Aim 3 showed that increasing the number of rings and ring separation improves homogeneity, but that increasing ring separation and radius reduces field strength. Guided by this preceding analysis, ten distinct parameter combinations were selected for further study. The complete list, along with details for each configuration, is provided in the Appendix. This manual approach allowed us to explore specific interactions and trade-offs between the parameters, such as testing a "wide radii" configuration against a "constant radii" one.

The tested combinations are detailed in the table in Appendix A. The following results from these combinations are presented below.

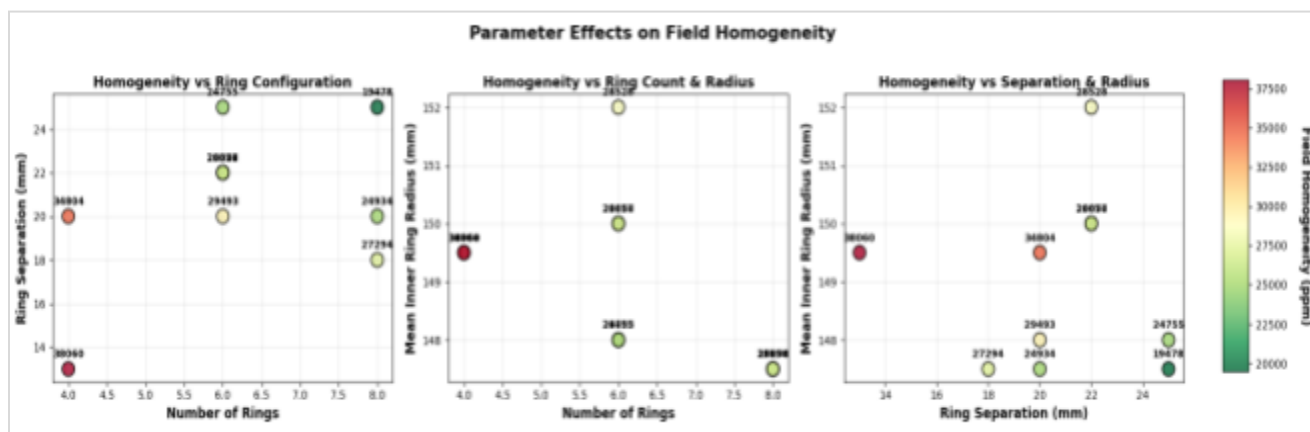


Figure 16: Comparison of field homogeneity (ppm) across different combinations of simulated Halbach

array parameters: number of rings, ring separation, and mean inner ring radius. Each point represents a specific parameter configuration, with color indicating simulated homogeneity (green = better homogeneity, red = worse).

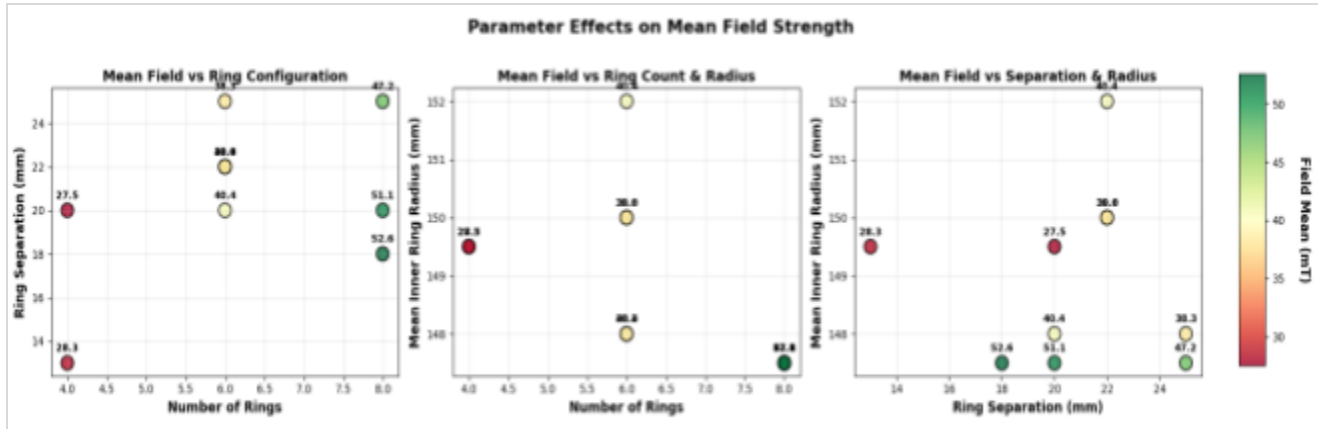


Figure 17: Comparison of field strength (mT) across different combinations of simulated Halbach array parameters: number of rings, ring separation, and mean inner ring radius. Each point represents a specific parameter configuration, with color indicating simulated mean (green = better; red = worse).

In the two pair-plot figures above, three pairwise parameter relationships are shown:

- Ring separation vs. number of rings (left)
- Mean inner ring radius vs. number of rings (center)
- Mean inner ring radius vs. ring separation (right)

For the graph on the far left, the simulated homogeneity is generally best (i.e., least spatial variation shown by greener points) when both the number of rings and the ring separation are relatively small, and it becomes poorest (redder points) as both parameters increase. However, no clear linear trend is explicitly present. Several configurations with low ring count and low separation still show moderate or poor homogeneity, indicating that the interaction between these parameters is not purely linear. Nonetheless, the best-performing homogeneity values appear across the full range of ring counts and separations, with a loose trend suggesting gradual improvement as these parameters increase. When comparing this with the corresponding field strength plot, a similar but more interpretable pattern emerges where increasing ring count tends to increase mean field strength, while larger ring separation generally reduces the field strength. This is consistent with the expectation that magnets placed farther apart produce weaker composite fields. Additionally, certain configurations that produce better homogeneity do not necessarily produce strong fields, highlighting a design trade-off between maximizing uniformity and maintaining high field strength. Overall, this left-most parameter pair (ring count and ring separation) shows the clearest relationship for both metrics, making it the most predictable of the three tested combinations.

The middle plot, which compares homogeneity against the number of rings and the mean inner ring radius, has trends that are much less clear. Good and bad homogeneity values are distributed irregularly across this parameter space. The sparse sampling of radius values, particularly compared to the ring-separation parameter which is more densely varied, makes interpretation difficult. The scattered pattern suggests that the relationship between radius and homogeneity may be nonlinear or influenced by other parameters not independently altered in this analysis. Capturing the underlying structure may require non-parametric modeling techniques, such as support vector decision boundaries, to map regions of optimal configurations and help guide users in the selection of these parameters. Additional sampling, especially with finer variations in radius, would significantly improve interpretability. The associated field strength plot helps

show the uncertainty that field strength has any dependency on radius, with neither strictly decreasing nor increasing as radius changes. Among the three plots, this configuration displays the weakest and least predictable relationship for both homogeneity and field strength.

In the rightmost homogeneity plot, comparing ring separation and mean inner radius, the configurations with better performance tend to occur at the moderate to high ring separation values, even though the mean inner radius remains relatively small. Alternatively, the worst homogeneity performances appear at lower ring separations, with less regard to the radius. This implies that ring separation is more consistent and influential on homogeneity compared to inner radius within the ranges tested. The trend is not linear, but visually, separation is the dominant parameter when compared to radius. Field strength behavior in this same pair-plot displays the expected opposing trend: that increasing ring separation generally decreases the mean field strength, while reducing separation tends to increase the field. However, the influence of mean inner radius on field strength is again weak and inconsistent. This reinforces the interpretation that ring separation is a primary driver for both homogeneity and strength, but in opposite directions. In other words, configurations yielding the best homogeneity in this plot frequently shows the reduced field strength, which further indicates the design trade-off between field uniformity and amplitude with these parameters, as was touched on in the previous two sections. Among the three sets of parameter combinations, this pair most clearly illustrates that trade-off as expected.

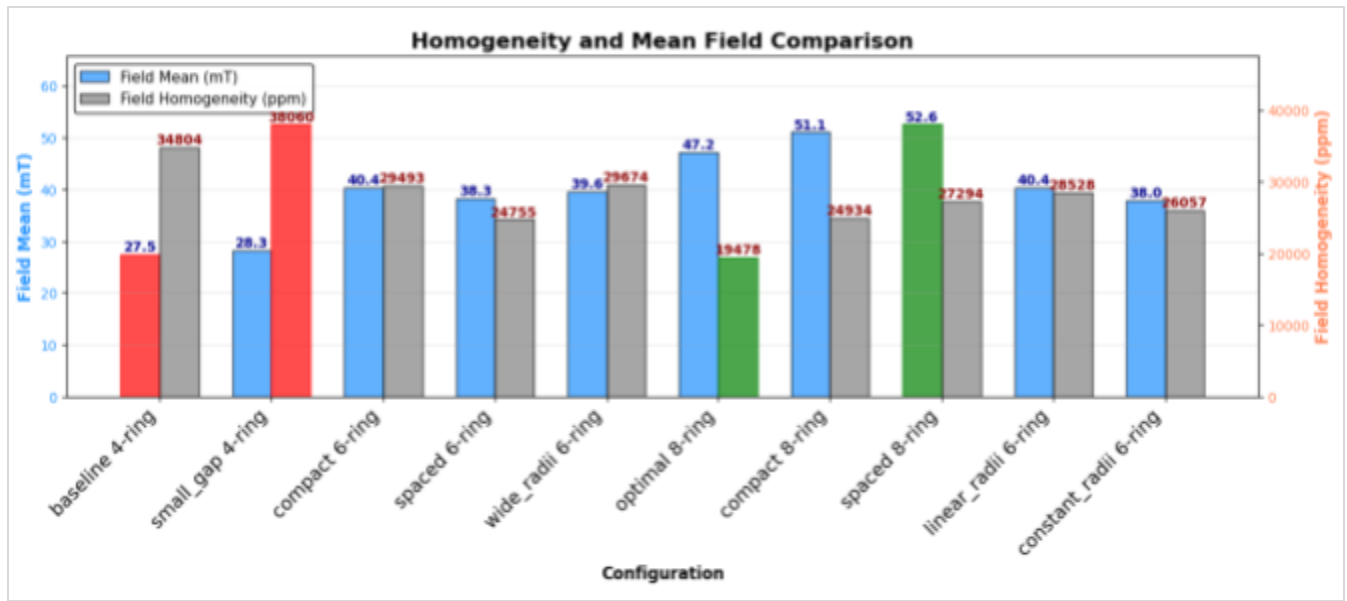


Figure 18: Final field mean and homogeneity for each pass of parameter combinations.

The figure above shows different combinations of the amount of rings, mean ring radius, and ring separation. The most optimal combination is the “optimal 8-ring” and the least optimal is the “small_gap 4-ring.” This figure demonstrates how an increase in the amount of rings and the gap size tend to have the greatest effects on improving homogeneity given the parameters we tested.

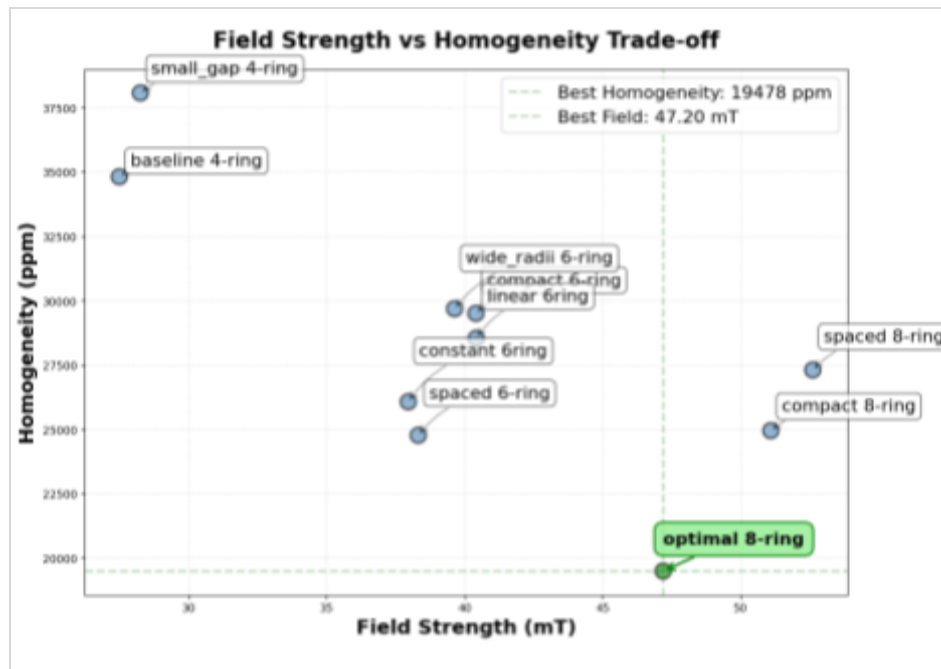


Figure 19: Field strength (mT) vs. homogeneity (ppm) for each pass of parameter combinations.

The figure above shows the differences between field strength and homogeneity given our combinations, the best ratio ends up being the “optimal 8-ring” as well since the field strength remains high and the homogeneity remains low. Notably, the “spaced 8-ring” and “compact 8-ring” also show favorable results, which align with our analysis of an increase in the number of rings causing singularly positive results in our first section’s analysis. It also appears that with each combination of parameters tested, the results in this figure tended to cluster together based primarily on the number of rings assigned in each simulation.

Conclusion:

Interpretation of Results

Given our results, we have come to the conclusion that an increase in the number of rings (Aim 1), mean ring radius (Aim 2), and ring separation (Aim 3), respectively, will create an overall improvement in homogeneity. Additionally, as the number of rings increases, field strength improves as well. However, the field strength decreases when mean ring radius and ring separation increase. That is, to summarize:

	Number of Rings ↑	Ring Radii ↑	Ring Separation ↑
Homogeneity	Improves (↓ ppm)	Improves (↓ ppm)	Improves (↓ ppm)
Field Strength	Improves (↑ mT)	Worsens (↓ mT)	Worsens (↓ mT)

From this, we can tell that adding more rings to a cylindrical Halbach MRI will only result in net positive changes. However, these improvements have diminishing returns, and physical implementation techniques can provide limitations to the number of rings that can actually be used in practice.

For the other two tests, both increasing the ring radii and separation, there is an important trade-off to take into account. Similar to increasing the number of rings, the increases for these two tests improved the homogeneity as well, but it actually decreased the field strength. This is problematic for implementation since both a low field homogeneity and high field strength are necessary for usable MRI systems. Therefore,

in the adjustment of these two parameters, this trade-off needs to be carefully monitored in order to optimize performance.

We can conclude that there must be a balance between each variable in order to find optimal combinations (Aim 4), one of which we believe is the “optimal 8-ring” combination where there are 8 rings, the inner ring radii are [140 mm, 145 mm, 150 mm, 155 mm] and the ring separation is 25 mm. Among the combinations we tested, this provides the best field strength to homogeneity ratio.

Overall, this study confirms that a cylindrical Halbach array can be tuned to significantly improve field homogeneity while maintaining usable field strength, primarily through increasing the number of rings while carefully balancing ring radii and separation.

Next Steps

One of the next steps for translating these simulation-based findings into practical designs for targeted imaging would be to explore the strategic removal of magnet rings to create intentional gaps in the array. This approach would allow us to alter the spatial focus of the magnetic field, potentially creating a stronger, more homogeneous field for a specific region. For example, this would assist with optimizing a field for imaging the chest wall between breast tissue and the rib cage.

Additionally, there would be value in creating a procedure for automatically creating physically feasible magnet combinations, incorporating constraints like minimum/magnet spacing and total device size, which will be defined in collaboration with the manufacturing team. Currently, the fields compared in our optimization section (Aim 4) were manually inputted and simulated. With the collaboration of the team working on printing and assembling these portable MRIs, helping to define the edge cases and other interactions would further assist in this task.

There is also a longer-term goal to create a graphic user interface (GUI) where a user can visually add or remove rings, adjust gaps and radii, and immediately see the resulting impact on field strength and homogeneity in real time using a 3-dimensional interactive simulation. This will allow for more quick and intuitive prototyping of Halbach arrays for specific tasks, such as the focus on the chest wall in a breast MRI.

Bibliography

- [1] P. Blümler, "Proposal for a permanent magnet system with a constant gradient mechanically adjustable in direction and strength," *Concepts in Magnetic Resonance Part B: Magnetic Resonance Engineering*, vol. 46, no. 1, pp. 41–48, Feb. 2016, doi: <https://doi.org/10.1002/cmr.b.21320>.
- [2] T. O'Reilly et al., LUMC-LowFieldMRI, "HalbachOptimisation/halbachFields.py at master · LUMC-LowFieldMRI/HalbachOptimisation," *GitHub*, 2025.
<https://github.com/LUMC-LoiwFieldMRI/HalbachOptimisation/blob/master/halbachFields.py> (accessed Nov. 23, 2025)

Appendix A

Program Files

The Python code is attached. A list of the files is as follows:

- **halbach_MRI_Parameters.ipynb**: code for analysis in Python. Run this file to alter simulation inputs.
- **halbachFields.py**: code for Halbach mathematical calculations, automatically imported into *halbach_MRI_Parameters.ipynb* for the simulation.
- **Saved passes of parameter combinations**: files with results, loaded in to save time on multiple repeated executions.
 - numrings_constant_sweep_full.pkl/numrings_constant_sweep_results.csv
 - numrings_linear_sweep_full.pkl/numrings_linear_sweep_results.csv
 - radii_sweep_full.pkl/radii_sweep_results.csv
 - gap_sweep_full.pkl/gap_sweep_results.csv
 - main_params_sweep_full.pkl/main_params_sweep_results.csv
- **README.txt**: markdown file with instructions to install necessary libraries to run *halbach_MRI_Parameters.ipynb*.
- **requirements.txt**: list of Python libraries to install in order to run *halbach_MRI_Parameters.ipynb*, as outlined in the *README.txt* file.

List of Parameter Combination Details

	Number of Rings	Inner Ring Radii (mm)	Inner Number of Magnets	Ring Separation (mm)
<i>baseline 4-ring</i>	4	[148, 151]	[50, 51]	20
<i>small_gap 4-ring</i>	4	[148, 151]	[50, 51]	13
<i>compact 6-ring</i>	6	[145, 148, 151]	[50, 51, 52]	20
<i>spaced 6-ring</i>	6	[145, 148, 151]	[50, 51, 52]	25
<i>wide_radii 6-ring</i>	6	[140, 150, 160]	[48, 52, 56]	22
<i>linear_radii 6-ring</i>	6	[142, 146, 150, 154, 158, 162]	[49, 50, 51, 52, 53, 54]	22
<i>constant_radii 6-ring</i>	6	[150, 150, 150]	[51, 51, 51]	22
<i>optimal 8-ring</i>	8	[140, 145, 150, 155]	[48, 50, 52, 54]	25
<i>compact 8-ring</i>	8	[140, 145, 150, 155]	[48, 50, 52, 54]	20
<i>spaced 8-ring</i>	8	[140, 145, 150, 155]	[48, 50, 52, 54]	18

Appendix B

Hard Copy of Code

```
import numpy as np
import halbachFields
import matplotlib
import matplotlib.pyplot as plt
import pandas as pd
import csv
import os
import pickle

"""
Based on homogeneityOptimization.py (Modified to isolate forward field calculations, RG 2025)

Created on Mon Sep 24 16:17:41 2018

@author: to_reilly
"""

def graph_fields(coordinateAxis, maskedField, shimmedField):
    """
    Tom O'Reilly's original code for graphing resulting simulated fields,
    included as a function with field parameters passed in.
    Modified version by Robert Griffin.

    Visualize various slices of the simulated magnetic field.

    coordinateAxis: magnetic field magnitude after applying the cylindrical mask.
    shimmedField : Full simulated magnetic field (in Tesla) before masking.

    Displays figures, no output
    """
    subFull, Fullax = plt.subplots(1, 3, figsize=(20, 5))
    Fullax[0].plot(coordinateAxis * 1e2,
                   maskedField[int(np.floor(np.size(maskedField, 0) / 2)),
                               int(np.floor(np.size(maskedField, 1) / 2)), :]+ 0.005)
    Fullax[1].plot(coordinateAxis * 1e2,
                   maskedField[int(np.floor(np.size(maskedField, 0) / 2)), :,
                               int(np.floor(np.size(maskedField, 2) / 2))])
    Fullax[2].plot(coordinateAxis * 1e2,
                   maskedField[:, int(np.floor(np.size(maskedField, 1) / 2)),
                               int(np.floor(np.size(maskedField, 2) / 2))])

    Fullax[0].set_xlabel('X axis (cm)')
    Fullax[0].set_ylabel('Field strength (Tesla)')
    Fullax[1].set_xlabel('Z axis (cm)')
    Fullax[1].set_ylabel('Field strength (Tesla)')
    Fullax[2].set_xlabel('Y axis (cm)')
    Fullax[2].set_ylabel('Field strength (Tesla)')
    plt.tight_layout()

    fig, ax = plt.subplots(1, 3)
    im1 = ax[0].imshow(maskedField[:, :, int(np.floor(np.size(maskedField, 2) / 2))])
    im2 = ax[1].imshow(maskedField[:, int(np.floor(np.size(maskedField, 1) / 2)), :])
    im3 = ax[2].imshow(maskedField[int(np.floor(np.size(maskedField, 0) / 2)), :, :])
    ax[0].set_xlabel('Z (mm)')
    ax[0].set_ylabel('Y (mm)')
    ax[1].set_xlabel('X (mm)')
    ax[1].set_ylabel('Y (mm)')
    ax[2].set_xlabel('X (mm)')
    ax[2].set_ylabel('Z (mm)')

    cbar1 = fig.colorbar(im1, ax=ax[0])
```

```

cbar2 = fig.colorbar(im2, ax=ax[1])
cbar3 = fig.colorbar(im3, ax=ax[2])

figu, axe = plt.subplots(1, 3)
i1 = axe[0].imshow(shimmedField[:, :, int(np.floor(np.size(shimmedField, 2) / 2))])
i2 = axe[1].imshow(shimmedField[:, int(np.floor(np.size(shimmedField, 1) / 2)), :])
i3 = axe[2].imshow(shimmedField[int(np.floor(np.size(shimmedField, 0) / 2)), :, :])
axe[0].set_xlabel('Z (mm)')
axe[0].set_ylabel('Y (mm)')
axe[1].set_xlabel('X (mm)')
axe[1].set_ylabel('Y (mm)')
axe[2].set_xlabel('X (mm)')
axe[2].set_ylabel('Z (mm)')

cb1 = figu.colorbar(i1, ax=axe[0])
cb2 = figu.colorbar(i2, ax=axe[1])
cb3 = figu.colorbar(i3, ax=axe[2])

plt.show()

```

Simulating the Halbach Array MRI (with ****one** set** of input parameters):

```

def halbach_array_simulation(numRings, innerRingRadii, innerNumMagnets, ringSep, graph):
    '''
    Tom O'Reilly's original code for simulating fields,
    included as a function with field parameters passed in.
    Modified version by Robert Griffin.

    Run a single Halbach array MRI field simulation using the given design parameters.

    This function builds the magnet array geometry (ring positions, radii, magnet counts),
    constructs a cylindrical mask defining the field-of-view,
    computes the 3D magnetic field using halbachFields.createHalbach(),
    applies the mask and computes mean field magnitude and homogeneity, and
    can optionally visualize the field distribution

    numRings: Number of magnet rings (symmetric about x = 0)
    innerRingRadii radii (in meters) for the inner Halbach rings
    innerNumMagnets: Number of magnets in each ring
    ringSep Spacial separation between adjacent rings (in mm)
    graph: Plots field slices and cross-sections if true

    Outputs:
    field_details:
    - 'Shimmed mean' (mT)
    - 'Shimmed homogeneity (mt)'
    - 'Shimmed homogeneity (ppm)'
    maskedField: Masked field magnitude used to compute homogeneity
    shimmedField: Full unmasked magnetic field (Bx aligned component)
    coordinateAxis: Axis coordinates (in meters) for the field volume
    '''
    ##### Define Magnet Array #####
    magnetLength = (numRings - 1) * ringSep * 1e-3
    # After using the gap size to calculate the total length of the magnet, determine the position of
    the center of each magnet ring (evenly spaced)
    ringPositions = np.linspace(-magnetLength / 2, magnetLength / 2, numRings)

    resolution = 1 # voxel size in mm
    DSV = 80 # Size of the unmasked simulation region in mm
    # Make sure that the total simulated field fits into the main memory of your computer
    # (reduce DSV or increase voxel size to reduce memory usage) or this will code will run extremely
    slow

    CSVname = "3D_Field.csv" # Name for the CSV file storing 3D field values resulting from
    the magnet simulation
    #graph = True # Toggle for plotting code

```

```

single_ring = False # This parameter allows for the addition of a larger magnet ring
that is concentric with the original ring

if single_ring:
    outerRingRadii = innerRingRadii
    outerNumMagnets = innerNumMagnets
else:
    outerRingRadii = innerRingRadii + 21 * 1e-3
    outerNumMagnets = innerNumMagnets + 7

##### Create Cylindrical mask
#####
# Field properties of the simulated magnet are calculated over a cylindrical volume around its
center
eval_Radius = 20 # Radius of the field evaluation region
eval_half_Thickness = 10 # Thickness of the field evaluation region

simDimensions = (DSV * 1e-3, DSV * 1e-3, DSV * 1e-3)

coordinateAxis = np.linspace(-simDimensions[0] / 2, simDimensions[0] / 2,
                             int(1e3 * simDimensions[0] / resolution + 1))
coords = np.meshgrid(coordinateAxis, coordinateAxis, coordinateAxis)

mask = np.zeros(np.shape(coords[0])) # Initialize an empty mask
mask[np.square(coords[0]) + np.square(coords[1]) <= (eval_Radius * 1e-3) ** 2] = 1 # Mark all
voxels within the cylindrical mask radius
mask[np.absolute(coords[2]) >= (eval_half_Thickness * 1e-3)+.001] = 0 # Unmark voxels
outside cylindrical mask thickness

##### Magnet Simulation
#####
ringPositionsSymmetry = ringPositions[ringPositions >= 0] # Remove negative ring positions.
Because this script assumes a symmetric magnet,
# negative ring positions will have a corresponding non-negative duplicate that can be used
instead

shimmedField = np.zeros(np.shape(mask) + (3,))
for positionIdx, position in enumerate(ringPositionsSymmetry): #Loop over every non-negative
ring position
    if position == 0:
        rings = (0,) #If a ring is positioned at x = 0, simulate only that ring
    else:
        rings = (-position, position) # If a ring is positioned away from x = 0, simulate both
that ring and the version reflected over the yz plane at x = 0

    # Call createHalbach() to simulate the field of a magnet ring based on:
    # the number of magnets in the ring(s) (numMagnets),
    # the x position of the ring(s) (rings),
    # the radius of the ring(s) (radius),
    # the size of the permanent magnets (magnetSize),
    # the resolution of the simulation (resolution),
    # and the size of the total simulation area (simDimensions)
    shimmedField += halbachFields.createHalbach(numMagnets=innerNumMagnets[positionIdx],
rings=rings,
radius=innerRingRadii[positionIdx],
magnetSize=0.012,
resolution=1e3 / resolution,
simDimensions=simDimensions)
    if not single_ring: # If there are multiple rings at the same x position, simulate the
field of the outer ring
        # using new values for numMagnets and radius
        shimmedField += halbachFields.createHalbach(numMagnets=outerNumMagnets[positionIdx],
rings=rings,

```

```

radius=outerRingRadii[positionIdx],

magnetSize=0.012,

resolution=1e3 / resolution,

simDimensions=simDimensions)

# Save 3D field values to a CSV file
Nx, Ny, Nz, _ = shimmedField.shape
with open(CSVname, mode="w", newline="") as f:
    writer = csv.writer(f)
    writer.writerow(["x", "y", "z", "Bx", "By", "Bz"])

    for i in range(Nx):
        for j in range(Ny):
            for k in range(Nz):
                Bx, By, Bz = shimmedField[i, j, k]
                writer.writerow([k, j, i, Bz, By, Bx])

shimmedField = shimmedField[..., 0]

#Apply the Cylindrical mask to the simulated magnetic field
mask[mask == 0] = np.nan
maskedField = np.abs(np.multiply(shimmedField, mask))

#Graph
if graph:
    graph_fields(coordinateAxis, maskedField, shimmedField)

#Print an overview of the magnet's field characteristics
print("Shimmed mean: %.2f mT" % (1e3 * np.nanmean(maskedField)))
print("Shimmed homogeneity: %.4f mT" % (1e3 * (np.nanmax(maskedField) - np.nanmin(maskedField))))
print("Shimmed homogeneity: %i ppm" % (
    1e6 * ((np.nanmax(maskedField) - np.nanmin(maskedField)) / np.nanmean(maskedField))))

field_details = {
    "Shimmed mean" : 1e3 * np.nanmean(maskedField),
    "Shimmed homogeneity (mt)" : 1e3 * (np.nanmax(maskedField) - np.nanmin(maskedField)),
    "Shimmed homogeneity (ppm)" : 1e6 * ((np.nanmax(maskedField) - np.nanmin(maskedField)) /
np.nanmean(maskedField))
}

#return field_details
return field_details, maskedField, shimmedField, coordinateAxis
Testing the Simulation with Original Parameters:
result = halbach_array_simulation(4, np.array([148, 151]) * 1e-3, np.array([50, 51]), 15, True)
Saving & Loading Results (Just to save time, takes ~15 minutes to execute all cells otherwise):
def save_results(results_list, field_data_list, filename_base):
    """Save simulation results to files"""
    # Save results as CSV for easy viewing
    results_df = pd.DataFrame([
        'config_name': r.get('config_name', r.get('name', 'Unknown')),
        'numRings': r['numRings'],
        'ringSep': r['ringSep'],
        'mean_field_mT': r['field_details']['Shimmed mean'],
        'homogeneity_mT': r['field_details']['Shimmed homogeneity (mt)'],
        'homogeneity_ppm': r['field_details']['Shimmed homogeneity (ppm)']
    ] for r in results_list)
    results_df.to_csv(f"{filename_base}_results.csv", index=False)

# Save full results including field data as pickle for complete reconstruction
with open(f"{filename_base}_full.pkl", 'wb') as f:
    pickle.dump({'results': results_list, 'fields': field_data_list}, f)

print(f"Results saved to {filename_base}_results.csv and {filename_base}_full.pkl")

def load_results(filename_base):

```

```

"""Load simulation results from files"""
pkl_file = f"{filename_base}_full.pkl"
if os.path.exists(pkl_file):
    with open(pkl_file, 'rb') as f:
        data = pickle.load(f)
    print(f"Loaded results from {pkl_file}")
    return data['results'], data['fields']
return None, None

## Plotting & Simulation (using **List** of Parameters) Functions
---
Running Halbach Simulation for each list of parameters in input:
def run_parameter_sweep(halbach_simulation_func, param_configs, show_progress=True,
save_fields=True):
    """
    Execute a batch of Halbach array simulations over multiple parameter sets.

    halbach_simulation_func: Simulation function to call for each parameter configuration
    param_configs:
    - 'numRings'
    - 'innerRingRadii'
    - 'innerNumMagnets'
    - 'ringSep'
    and optionally 'config_name'.
    show_progress: prints progress updates and computed metrics during the sweep if true
    save_fields: stores masked field data and coordinate axis for each simulation if true

    outputs:
    results_list: Summary of parameters and computed field metrics for each configuration
    field_data_list: (maskedField, coordinateAxis, field_details) tuple if save_fields
    """
    results_list = []
    field_data_list = [] if save_fields else None

    for i, config in enumerate(param_configs):
        # Handle both 'name' and 'config_name' keys
        config_name = config.get('config_name', config.get('name', f'Config_{i+1}'))

        if show_progress:
            print(f"\nRunning simulation {i+1}/{len(param_configs)}: {config_name}")

        # Run simulation with graph=False to suppress individual plots
        result = halbach_simulation_func(
            numRings=config['numRings'],
            innerRingRadii=config['innerRingRadii'],
            innerNumMagnets=config['innerNumMagnets'],
            ringSep=config['ringSep'],
            graph=False
        )

        # Unpack results
        if len(result) == 4:
            field_details, maskedField, shimmedField, coordinateAxis = result

            # Save field data if requested
            if save_fields:
                field_data_list.append((maskedField, coordinateAxis, field_details))
        else:
            field_details = result

        # Calculate innerRingRadii_mm if not provided
        innerRingRadii_mm = config.get('innerRingRadii_mm', config['innerRingRadii'] * 1e3)

        # Store results with parameters
        result_entry = {
            'numRings': config['numRings'],

```

```

        'innerRingRadii': config['innerRingRadii'],
        'innerRingRadii_mm': innerRingRadii_mm,
        'innerNumMagnets': config['innerNumMagnets'],
        'ringSep': config['ringSep'],
        'config_name': config_name,
        'field_details': field_details
    }

    results_list.append(result_entry)

    if show_progress:
        print(f"  Mean: {field_details['Field mean']:.2f} mT, "
              f"  Homogeneity: {field_details['Field homogeneity (ppm)']:.0f} ppm")

print(f"\n{'='*60}")
print(f"Parameter sweep complete! {len(results_list)} simulations finished.")
print(f"{'='*60}")

# Find best configuration
best_idx = np.argmin([r['field_details']['Field homogeneity (ppm)']
                     for r in results_list])
best_config = results_list[best_idx]
print(f"\nBest configuration: {best_config['config_name']}")
print(f"  Homogeneity: {best_config['field_details']['Field homogeneity (ppm)']:.0f} ppm")
print(f"  Mean field: {best_config['field_details']['Field mean']:.2f} mT")

if save_fields:
    return results_list, field_data_list
else:
    return results_list

Visualizing results:
def plot_field_metrics_histogram(all_results):
    """
    Creates a grouped bar chart showing both Shimmed mean (mT) and
    Shimmed homogeneity (ppm) for each configuration.

    all_results: list of result dictionaries containing field details

    outputs:
    fig: figure containing grouped bar chart
    """
    # Extract values and names
    homogeneity_ppm = []
    shimmed_mean_mT = []
    names = []

    for r in all_results:
        # Check if it's from run_parameter_sweep (nested format)
        if 'field_details' in r:
            homogeneity_ppm.append(r['field_details']['Shimmed homogeneity (ppm)'])
            shimmed_mean_mT.append(r['field_details']['Shimmed mean'])
            names.append(r.get('config_name', r.get('name', 'Unknown')))
        # Or from run_simulation (direct format)
        else:
            homogeneity_ppm.append(r['homogeneity_ppm'])
            shimmed_mean_mT.append(r.get('shimmed_mean_mT', r.get('mean_field_mT', 0)))
            names.append(r['name'])

    # Create figure with two y-axes
    fig, ax1 = plt.subplots(figsize=(14, 6))

    x_pos = np.arange(len(names))
    width = 0.35 # Width of bars

    # Create first set of bars (Field mean) on left y-axis

```

```

bars1 = ax1.bar(x_pos - width/2, shimmed_mean_mT, width,
               label='Field Mean (mT)',
               color='dodgerblue', edgecolor='black', alpha=0.7)

# Color the best (highest) mean bar in green, worst in red
best_mean_idx = shimmed_mean_mT.index(max(shimmed_mean_mT))
worst_mean_idx = shimmed_mean_mT.index(min(shimmed_mean_mT))
bars1[best_mean_idx].set_color('green')
bars1[worst_mean_idx].set_color('red')

# Set up first y-axis
ax1.set_ylabel('Field Mean (mT)', fontsize=12, fontweight='bold', color='dodgerblue')
ax1.tick_params(axis='y', labelcolor='dodgerblue')
ax1.set_xlabel('Configuration', fontsize=12, fontweight='bold')

# Create second y-axis for homogeneity
ax2 = ax1.twinx()
bars2 = ax2.bar(x_pos + width/2, homogeneity_ppm, width,
               label='Field Homogeneity (ppm)',
               color='grey', edgecolor='black', alpha=0.7)

# Color the best (lowest) homogeneity bar in green, worst in red
best_homog_idx = homogeneity_ppm.index(min(homogeneity_ppm))
worst_homog_idx = homogeneity_ppm.index(max(homogeneity_ppm))
bars2[best_homog_idx].set_color('green')
bars2[worst_homog_idx].set_color('red')

# Set up second y-axis
ax2.set_ylabel('Field Homogeneity (ppm)', fontsize=12, fontweight='bold', color='coral')
ax2.tick_params(axis='y', labelcolor='coral')

# Set x-axis
ax1.set_xticks(x_pos)
ax1.set_xticklabels(names, rotation=45, ha='right', fontsize=14)
ax1.set_title('Homogeneity and Mean Field Comparison', fontsize=16, fontweight='bold')
ax1.grid(True, alpha=0.3, axis='y')

# Add value labels on bars
for bar, val in zip(bars1, shimmed_mean_mT):
    height = bar.get_height()
    ax1.text(bar.get_x() + bar.get_width()/2, height,
            f'{val:.1f}', ha='center', va='bottom',
            fontsize=10, fontweight='bold', color='darkblue')

for bar, val in zip(bars2, homogeneity_ppm):
    height = bar.get_height()
    ax2.text(bar.get_x() + bar.get_width()/2, height,
            f'{val:.0f}', ha='center', va='bottom',
            fontsize=10, fontweight='bold', color='darkred')

# Create proxy artists for legend with correct colors
from matplotlib.patches import Patch
legend_elements = [
    Patch(facecolor='dodgerblue', edgecolor='black', alpha=0.7, label='Field Mean (mT)'),
    Patch(facecolor='grey', edgecolor='black', alpha=0.7, label='Field Homogeneity (ppm)')
]

# Add legend with proxy artists
ax1.legend(handles=legend_elements, loc='upper left', fontsize=11,
          framealpha=0.95, edgecolor='black')

# Adjust y-axis limits to create space at top for legend and bar labels
y1_max = max(shimmed_mean_mT)
y2_max = max(homogeneity_ppm)
ax1.set_ylim(0, y1_max * 1.25) # Add 25% space at top

```



```

ax2.set_ylim(0, y2_max * 1.25) # Add 25% space at top

fig.tight_layout()
fig.subplots_adjust(top=0.92) # Add space at top for legend
return fig
def param_homogeneity_trendline(all_results, param, param_label, degree, param_extractor=None):
    """
    Plot homogeneity versus a single design parameter with regression trendline.

    all_results: results from `run_parameter_sweep()`
    param_name: Display name of current parameter
    param_extractor: Function mapping a result dict; numeric parameter value
    order: Order for regression line (1 for linear, 2 for quadratic)

    outputs:
    fig: Figure containing the scatter plots and regression curves.
    """
    # Create a line plot with trendline showing gap vs homogeneity
    fig, ax = plt.subplots(figsize=(10, 6))

    # Extract parameter values - handle arrays
    params_raw = [r[param] for r in all_results]

    # If param_extractor is provided, use it; otherwise use mean for arrays
    if param_extractor is None:
        params = np.array([np.mean(p) if isinstance(p, np.ndarray) else p
                           for p in params_raw]) # Convert to mm if array
    else:
        params = np.array([param_extractor(p) for p in params_raw])

    homogeneity = np.array([r['field_details']['Shimmed homogeneity (ppm)'] for r in all_results])

    # Plot actual data
    ax.plot(params, homogeneity, marker='o', linewidth=2, markersize=8,
            color='steelblue', label='Measured', zorder=3)

    # Fit polynomial trendline using degree given:
    z = np.polyfit(params, homogeneity, degree)
    p = np.poly1d(z)

    # Generate smooth line for trendline
    params_smooth = np.linspace(params.min(), params.max(), 100)
    homogeneity_trend = p(params_smooth)

    # Plotting the trendline
    ax.plot(params_smooth, homogeneity_trend, '--', linewidth=2,
            color='red', alpha=0.7, label='Trendline', zorder=2)

    # Calculate R²
    homogeneity_pred = p(params)
    ss_res = np.sum((homogeneity - homogeneity_pred) ** 2)
    ss_tot = np.sum((homogeneity - np.mean(homogeneity)) ** 2)
    r_squared = 1 - (ss_res / ss_tot)

    # Format equation based on degree
    if degree == 1:
        equation_text = f'y = {z[0]:.2f}x + {z[1]:.2f}'
    else:
        equation_text = f'y = {z[0]:.2f}x² + {z[1]:.2f}x + {z[2]:.2f}'

    ax.text(0.04, 0.10, equation_text, transform=ax.transAxes,
            fontsize=16, verticalalignment='top',
            bbox=dict(boxstyle='round', facecolor='wheat', alpha=0.8))

    ax.set_xlabel(param_label, fontsize=12, fontweight='bold')

```

```

ax.set_ylabel('Homogeneity (ppm)', fontsize=12, fontweight='bold')
ax.set_title(f'Effect of {param_label} on Field Homogeneity', fontsize=14, fontweight='bold')
ax.grid(True, alpha=0.3)

# Add value labels
for i, h in zip(params, homogeneity):
    ax.text(i, h+200, f'{h:.0f}', ha='center', va='bottom', fontsize=12)

ax.legend(fontsize=11, loc='best')
plt.tight_layout()
plt.show()
print(f'\tR2 = {r_squared:.4f}')
def param_field_strength_trendline(all_results, param, param_label, degree, param_extractor=None):
    """
    Plot field strength versus a single design parameter with regression trendline.

    all_results: results from `run_parameter_sweep()`
    param_name: Display name of current parameter
    param_extractor: Function mapping a result dict; numeric parameter value
    order: Order for regression line (1 for linear, 2 for quadratic)

    outputs:
    fig: Figure containing the scatter plots and regression curves.
    """

    # Create a line plot with trendline showing gap vs field_strength
    fig, ax = plt.subplots(figsize=(10, 6))

    # Extract parameter values - handle arrays
    params_raw = [r[param] for r in all_results]

    # If param_extractor is provided, use it; otherwise use mean for arrays
    if param_extractor is None:
        params = np.array([np.mean(p) if isinstance(p, np.ndarray) else p
                           for p in params_raw])
    else:
        params = np.array([param_extractor(p) for p in params_raw])

    # Convert field strength to mT
    field_strength = np.array([r['field_details']['Shimmed mean'] for r in all_results])

    # Plot actual data
    ax.plot(params, field_strength, marker='o', linewidth=2, markersize=8,
            color='steelblue', label='Measured', zorder=3)

    # Fit polynomial trendline using degree given:
    z = np.polyfit(params, field_strength, degree)
    p = np.poly1d(z)

    # Generate smooth line for trendline
    params_smooth = np.linspace(params.min(), params.max(), 100)
    field_strength_trend = p(params_smooth)

    # Plotting the trendline
    ax.plot(params_smooth, field_strength_trend, '--', linewidth=2,
            color='red', alpha=0.7, label='Trendline', zorder=2)

    # Calculate R2
    field_strength_pred = p(params)
    ss_res = np.sum((field_strength - field_strength_pred) ** 2)
    ss_tot = np.sum((field_strength - np.mean(field_strength)) ** 2)
    r_squared = 1 - (ss_res / ss_tot)

    # Format equation based on degree
    if degree == 1:

```

```

    equation_text = f'y = {z[0]:.4f}x + {z[1]:.4f}'
else:
    equation_text = f'y = {z[0]:.4f}x2 + {z[1]:.4f}x + {z[2]:.4f}'

ax.text(0.04, 0.10, equation_text, transform=ax.transAxes,
        fontsize=16, verticalalignment='top',
        bbox=dict(boxstyle='round', facecolor='wheat', alpha=0.8))

ax.set_xlabel(param_label, fontsize=12, fontweight='bold')
ax.set_ylabel('Field Strength (mT)', fontsize=12, fontweight='bold')
ax.set_title(f'Effect of {param_label} on Field Strength', fontsize=14, fontweight='bold')
ax.grid(True, alpha=0.3)

# Add value labels with adaptive offset
y_range = field_strength.max() - field_strength.min()
offset = y_range * 0.02 # 2% of the range

for i, h in zip(params, field_strength):
    ax.text(i, h + offset, f'{h:.2f}', ha='center', va='bottom', fontsize=10)

ax.legend(fontsize=11, loc='best')
plt.tight_layout()
plt.show()
print(f'\tR2 = {r_squared:.4f}')
## **Altering Parameters:** Number of Rings
---
### First, keeping the array values **constant:**
numrings_constant_configs = []

for n in range(2, 20+1, 2):
    numrings_constant_configs.append({
        'name': f'{n}-ring (constant)',
        'numRings': n,
        'innerRingRadii': np.array([150] * int(n/2)) * 1e-3,
        'innerNumMagnets': np.array([51] * int(n/2)),
        'ringSep': 20
    })
# NumRings constant sweep
print("\nRunning numRings (constant radii) test sweep...")
numrings_const_results, numrings_const_field_data = load_results('numrings_constant_sweep')
if numrings_const_results is None:
    print("No saved results found. Running simulations...")
    numrings_const_results, numrings_const_field_data = run_parameter_sweep(
        halbach_array_simulation,
        numrings_constant_configs,
        show_progress=True,
        save_fields=True
    )
    save_results(numrings_const_results, numrings_const_field_data, 'numrings_constant_sweep')
else:
    print("Using previously-saved results.")
param_homogeneity_trendline(numrings_const_results, 'numRings', 'Number of Rings (Constant)', 1)
param_homogeneity_trendline(numrings_const_results, 'numRings', 'Number of Rings (Constant)', 2)

param_field_strength_trendline(numrings_const_results, 'numRings', 'Number of Rings (Constant)', 1)
param_field_strength_trendline(numrings_const_results, 'numRings', 'Number of Rings (Constant)', 2)

plot_field_metrics_histogram(numrings_const_results)
plt.show()
## **Altering Parameters:** Radii Size
---
radii_test_configs = [
    {
        'name': 'radii (140-143)',
        'numRings': 4,

```

```

        'innerRingRadii': np.array([140, 143]) * 1e-3,
        'innerNumMagnets': np.array([48, 49]),
        'ringSep': 20
    },
    {
        'name': 'radii (142-145)',
        'numRings': 4,
        'innerRingRadii': np.array([142, 145]) * 1e-3,
        'innerNumMagnets': np.array([49, 50]),
        'ringSep': 20
    },
    {
        'name': 'radii (144-147)',
        'numRings': 4,
        'innerRingRadii': np.array([144, 147]) * 1e-3,
        'innerNumMagnets': np.array([49, 50]),
        'ringSep': 20
    },
    {
        'name': 'radii (146-149)',
        'numRings': 4,
        'innerRingRadii': np.array([146, 149]) * 1e-3,
        'innerNumMagnets': np.array([50, 51]),
        'ringSep': 20
    },
    {
        'name': 'radii (148-151)', # Baseline
        'numRings': 4,
        'innerRingRadii': np.array([148, 151]) * 1e-3,
        'innerNumMagnets': np.array([50, 51]),
        'ringSep': 20
    },
    {
        'name': 'radii (150-153)',
        'numRings': 4,
        'innerRingRadii': np.array([150, 153]) * 1e-3,
        'innerNumMagnets': np.array([51, 52]),
        'ringSep': 20
    },
    {
        'name': 'radii (152-155)',
        'numRings': 4,
        'innerRingRadii': np.array([152, 155]) * 1e-3,
        'innerNumMagnets': np.array([52, 53]),
        'ringSep': 20
    },
    {
        'name': 'radii (154-157)',
        'numRings': 4,
        'innerRingRadii': np.array([154, 157]) * 1e-3,
        'innerNumMagnets': np.array([53, 54]),
        'ringSep': 20
    },
    {
        'name': 'radii (156-159)',
        'numRings': 4,
        'innerRingRadii': np.array([156, 159]) * 1e-3,
        'innerNumMagnets': np.array([53, 54]),
        'ringSep': 20
    }
]

print("Running radii test sweep...")
radii_results, radii_field_data = load_results('radii_sweep')
if radii_results is None:
    print("No saved results found. Running simulations...")

```

```

radii_results, radii_field_data = run_parameter_sweep(
    halbach_array_simulation,
    radii_test_configs,
    show_progress=True,
    save_fields=True
)
save_results(radii_results, radii_field_data, 'radii_sweep')
else:
    print("Using previously-saved results.")
param_homogeneity_trendline(radii_results, 'innerRingRadii', 'Mean Ring Radius (m)', 1)
param_homogeneity_trendline(radii_results, 'innerRingRadii', 'Mean Ring Radius (m)', 2)

param_field_strength_trendline(radii_results, 'innerRingRadii', 'Mean Ring Radius (m)', 1)
param_field_strength_trendline(radii_results, 'innerRingRadii', 'Mean Ring Radius (m)', 2)

plot_field_metrics_histogram(radii_results)
plt.show()
## **Altering Parameters:** Gap Size
---
gap_test_configs = [
    # Baseline configuration with varying gap sizes
    {
        'name': 'gap 12mm',
        'numRings': 4,
        'innerRingRadii': np.array([148, 151]) * 1e-3,
        'innerNumMagnets': np.array([50, 51]),
        'ringSep': 12 # Minimum reasonable/practical value
    },
    {
        'name': 'gap 13mm',
        'numRings': 4,
        'innerRingRadii': np.array([148, 151]) * 1e-3,
        'innerNumMagnets': np.array([50, 51]),
        'ringSep': 13
    },
    {
        'name': 'gap 14mm',
        'numRings': 4,
        'innerRingRadii': np.array([148, 151]) * 1e-3,
        'innerNumMagnets': np.array([50, 51]),
        'ringSep': 14
    },
    {
        'name': 'gap 15mm',
        'numRings': 4,
        'innerRingRadii': np.array([148, 151]) * 1e-3,
        'innerNumMagnets': np.array([50, 51]),
        'ringSep': 15
    },
    {
        'name': 'gap 16mm',
        'numRings': 4,
        'innerRingRadii': np.array([148, 151]) * 1e-3,
        'innerNumMagnets': np.array([50, 51]),
        'ringSep': 16
    },
    {
        'name': 'gap 17mm',
        'numRings': 4,
        'innerRingRadii': np.array([148, 151]) * 1e-3,
        'innerNumMagnets': np.array([50, 51]),
        'ringSep': 17
    },
    {
        'name': 'gap 18mm',

```

```

        'numRings': 4,
        'innerRingRadii': np.array([148, 151]) * 1e-3,
        'innerNumMagnets': np.array([50, 51]),
        'ringSep': 18
    },
    {
        'name': 'gap 19mm',
        'numRings': 4,
        'innerRingRadii': np.array([148, 151]) * 1e-3,
        'innerNumMagnets': np.array([50, 51]),
        'ringSep': 19
    },
    {
        'name': 'gap 20mm',
        'numRings': 4,
        'innerRingRadii': np.array([148, 151]) * 1e-3,
        'innerNumMagnets': np.array([50, 51]),
        'ringSep': 20 # Maximum reasonable/practical value
    },
    {
        'name': 'gap 21mm',
        'numRings': 4,
        'innerRingRadii': np.array([148, 151]) * 1e-3,
        'innerNumMagnets': np.array([50, 51]),
        'ringSep': 21
    },
    {
        'name': 'gap 22mm',
        'numRings': 4,
        'innerRingRadii': np.array([148, 151]) * 1e-3,
        'innerNumMagnets': np.array([50, 51]),
        'ringSep': 22
    },
    {
        'name': 'gap 23mm',
        'numRings': 4,
        'innerRingRadii': np.array([148, 151]) * 1e-3,
        'innerNumMagnets': np.array([50, 51]),
        'ringSep': 23
    },
    {
        'name': 'gap 24mm',
        'numRings': 4,
        'innerRingRadii': np.array([148, 151]) * 1e-3,
        'innerNumMagnets': np.array([50, 51]),
        'ringSep': 24
    },
    {
        'name': 'gap 25mm',
        'numRings': 4,
        'innerRingRadii': np.array([148, 151]) * 1e-3,
        'innerNumMagnets': np.array([50, 51]),
        'ringSep': 25
    }
]

print("\nRunning gap test sweep...")
gap_results, gap_field_data = load_results('gap_sweep')

if gap_results is None:
    print("No saved results found. Running simulations...")
    gap_results, gap_field_data = run_parameter_sweep(
        halbach_array_simulation,
        gap_test_configs,
        show_progress=True,
        save_fields=True
    )

```

```

    )
    save_results(gap_results, gap_field_data, 'gap_sweep')
else:
    print("Using previously-saved results.")
param_homogeneity_trendline(gap_results, 'ringSep', 'Ring Separation (mm)', 1)
param_homogeneity_trendline(gap_results, 'ringSep', 'Ring Separation (mm)', 2)

param_field_strength_trendline(gap_results, 'ringSep', 'Ring Separation (mm)', 1)
param_field_strength_trendline(gap_results, 'ringSep', 'Ring Separation (mm)', 2)

plot_field_metrics_histogram(gap_results)
plt.show()
<!-- ### Second, defining the array values **linearly:** -->
## **Analysis of Parameter Combinations**
---
**NOTE:** These parameter lists are not automatically/procedurally generated. These are just
different permutations of values we manually inputted.
# Running multiple simulations with different parameters, and SAVING THE RESULTS (for comparison
later)
different_param_passes = [
    # Baseline/original
    {
        'name': 'baseline 4-ring',
        'numRings': 4,
        'innerRingRadii': np.array([148, 151]) * 1e-3,
        'innerNumMagnets': np.array([50, 51]),
        'ringSep': 20
    },
    {
        'name': 'small_gap 4-ring',
        'numRings': 4,
        'innerRingRadii': np.array([148, 151]) * 1e-3,
        'innerNumMagnets': np.array([50, 51]),
        'ringSep': 13
    },
    {
        'name': 'compact 6-ring',
        'numRings': 6,
        'innerRingRadii': np.array([145, 148, 151]) * 1e-3,
        'innerNumMagnets': np.array([50, 51, 52]),
        'ringSep': 20
    },
    {
        'name': 'spaced 6-ring',
        'numRings': 6,
        'innerRingRadii': np.array([145, 148, 151]) * 1e-3,
        'innerNumMagnets': np.array([50, 51, 52]),
        'ringSep': 25
    },
    {
        'name': 'wide_radii 6-ring',
        'numRings': 6,
        'innerRingRadii': np.array([140, 150, 160]) * 1e-3,
        'innerNumMagnets': np.array([48, 52, 56]),
        'ringSep': 22
    },
    {
        'name': 'optimal 8-ring',
        'numRings': 8,
        'innerRingRadii': np.array([140, 145, 150, 155]) * 1e-3,
        'innerNumMagnets': np.array([48, 50, 52, 54]),
        'ringSep': 25
    },
    {
        'name': 'compact 8-ring',

```

```

        'numRings': 8,
        'innerRingRadii': np.array([140, 145, 150, 155]) * 1e-3,
        'innerNumMagnets': np.array([48, 50, 52, 54]),
        'ringSep': 20
    },
    {
        'name': 'spaced 8-ring',
        'numRings': 8,
        'innerRingRadii': np.array([140, 145, 150, 155]) * 1e-3,
        'innerNumMagnets': np.array([48, 50, 52, 54]),
        'ringSep': 18
    },
    {
        'name': 'linear_radii 6-ring',
        'numRings': 6,
        'innerRingRadii': np.array([142, 146, 150, 154, 158, 162]) * 1e-3,
        'innerNumMagnets': np.array([49, 50, 51, 52, 53, 54]),
        'ringSep': 22
    },
    {
        'name': 'constant_radii 6-ring',
        'numRings': 6,
        'innerRingRadii': np.array([150, 150, 150]) * 1e-3,
        'innerNumMagnets': np.array([51, 51, 51]),
        'ringSep': 22
    }
]

```

Running the simulation with each list of parameters:\

Using saved results if they exist, just to save some time...

```

def plot_parameter_pairs(results_list):
    """
    Creates two separate figures showing parameter relationships:
    Figure 1: Colored by Shimmed homogeneity (ppm), Figure 2: Colored by Shimmed mean (mT)

    Each figure contains three scatter plots showing
    * Number of Rings vs Ring Separation
    * Number of Rings vs Mean Inner Ring Radius
    * Ring Separation vs Mean Inner Ring Radius

    all_results: simulation results including 'numRings', 'innerRingRadii_mm', and 'ringSep'

    outputs:
    fig: Figure containing the parameter-pair scatterplot matrix
    """
    num_rings = []
    mean_radii = []
    ring_sep = []
    homogeneity_ppm = []
    shimmed_mean_mT = []

    for result in results_list:
        num_rings.append(result['numRings'])
        mean_radii.append(np.mean(result['innerRingRadii']) * 1e3)
        ring_sep.append(result['ringSep'])
        homogeneity_ppm.append(result['field_details']['Shimmed homogeneity (ppm)'])
        shimmed_mean_mT.append(result['field_details']['Shimmed mean'])

    # Convert to numpy arrays
    num_rings = np.array(num_rings)
    mean_radii = np.array(mean_radii)
    ring_sep = np.array(ring_sep)
    homogeneity_ppm = np.array(homogeneity_ppm)
    shimmed_mean_mT = np.array(shimmed_mean_mT)

    # ===== FIGURE 1: Homogeneity (ppm) =====

```



```

fig1, axes1 = plt.subplots(1, 3, figsize=(18, 5))

# Color mapping based on homogeneity (lower is better)
norm1 = plt.Normalize(vmin=homogeneity_ppm.min(), vmax=homogeneity_ppm.max())
cmap1 = matplotlib.cm.get_cmap('RdYlGn_r') # Red (bad) to Green (good), reversed

# Plot 1: numRings vs ringSep
scatter1_1 = axes1[0].scatter(num_rings, ring_sep,
                              c=homogeneity_ppm, s=150,
                              cmap=cmap1, norm=norm1,
                              edgecolors='black', linewidth=1.5, alpha=0.8)
axes1[0].set_xlabel('Number of Rings', fontsize=12, fontweight='bold')
axes1[0].set_ylabel('Ring Separation (mm)', fontsize=12, fontweight='bold')
axes1[0].set_title('Homogeneity vs Ring Configuration', fontsize=13, fontweight='bold')
axes1[0].grid(True, alpha=0.3)

for i, (x, y, h) in enumerate(zip(num_rings, ring_sep, homogeneity_ppm)):
    axes1[0].annotate(f'{h:.0f}', (x, y),
                      textcoords="offset points", xytext=(0,10),
                      ha='center', fontsize=9, fontweight='bold')

# Plot 2: numRings vs mean innerRingRadii
scatter1_2 = axes1[1].scatter(num_rings, mean_radai,
                              c=homogeneity_ppm, s=150,
                              cmap=cmap1, norm=norm1,
                              edgecolors='black', linewidth=1.5, alpha=0.8)
axes1[1].set_xlabel('Number of Rings', fontsize=12, fontweight='bold')
axes1[1].set_ylabel('Mean Inner Ring Radius (mm)', fontsize=12, fontweight='bold')
axes1[1].set_title('Homogeneity vs Ring Count & Radius', fontsize=13, fontweight='bold')
axes1[1].grid(True, alpha=0.3)

for i, (x, y, h) in enumerate(zip(num_rings, mean_radai, homogeneity_ppm)):
    axes1[1].annotate(f'{h:.0f}', (x, y),
                      textcoords="offset points", xytext=(0,10),
                      ha='center', fontsize=9, fontweight='bold')

# Plot 3: ringSep vs mean innerRingRadii
scatter1_3 = axes1[2].scatter(ring_sep, mean_radai,
                              c=homogeneity_ppm, s=150,
                              cmap=cmap1, norm=norm1,
                              edgecolors='black', linewidth=1.5, alpha=0.8)
axes1[2].set_xlabel('Ring Separation (mm)', fontsize=12, fontweight='bold')
axes1[2].set_ylabel('Mean Inner Ring Radius (mm)', fontsize=12, fontweight='bold')
axes1[2].set_title('Homogeneity vs Separation & Radius', fontsize=13, fontweight='bold')
axes1[2].grid(True, alpha=0.3)

for i, (x, y, h) in enumerate(zip(ring_sep, mean_radai, homogeneity_ppm)):
    axes1[2].annotate(f'{h:.0f}', (x, y),
                      textcoords="offset points", xytext=(0,10),
                      ha='center', fontsize=9, fontweight='bold')

# Add colorbar for Figure 1
fig1.subplots_adjust(right=0.92)
cbar_ax1 = fig1.add_axes([0.94, 0.15, 0.02, 0.7])
cbar1 = fig1.colorbar(scatter1_3, cax=cbar_ax1)
cbar1.set_label('Field Homogeneity (ppm)', rotation=270, labelpad=25,
                fontsize=12, fontweight='bold')

plt.suptitle('Parameter Effects on Field Homogeneity',
             fontsize=16, fontweight='bold', y=0.98)
plt.tight_layout(rect=[0, 0, 0.92, 0.96])

# ===== FIGURE 2: Shimmed Mean (mT) =====
fig2, axes2 = plt.subplots(1, 3, figsize=(18, 5))

```

```

# Color mapping based on shimmed mean (higher is better)
norm2 = plt.Normalize(vmin=shimmed_mean_mT.min(), vmax=shimmed_mean_mT.max())
cmap2 = matplotlib.cm.get_cmap('RdYlGn') # Red (low) to Green (high)

# Plot 1: numRings vs ringSep
scatter2_1 = axes2[0].scatter(num_rings, ring_sep,
                              c=shimmed_mean_mT, s=150,
                              cmap=cmap2, norm=norm2,
                              edgecolors='black', linewidth=1.5, alpha=0.8)
axes2[0].set_xlabel('Number of Rings', fontsize=12, fontweight='bold')
axes2[0].set_ylabel('Ring Separation (mm)', fontsize=12, fontweight='bold')
axes2[0].set_title('Mean Field vs Ring Configuration', fontsize=13, fontweight='bold')
axes2[0].grid(True, alpha=0.3)

for i, (x, y, m) in enumerate(zip(num_rings, ring_sep, shimmed_mean_mT)):
    axes2[0].annotate(f'{m:.1f}', (x, y),
                      textcoords="offset points", xytext=(0,10),
                      ha='center', fontsize=9, fontweight='bold')

# Plot 2: numRings vs mean innerRingRadii
scatter2_2 = axes2[1].scatter(num_rings, mean_radai,
                              c=shimmed_mean_mT, s=150,
                              cmap=cmap2, norm=norm2,
                              edgecolors='black', linewidth=1.5, alpha=0.8)
axes2[1].set_xlabel('Number of Rings', fontsize=12, fontweight='bold')
axes2[1].set_ylabel('Mean Inner Ring Radius (mm)', fontsize=12, fontweight='bold')
axes2[1].set_title('Mean Field vs Ring Count & Radius', fontsize=13, fontweight='bold')
axes2[1].grid(True, alpha=0.3)

for i, (x, y, m) in enumerate(zip(num_rings, mean_radai, shimmed_mean_mT)):
    axes2[1].annotate(f'{m:.1f}', (x, y),
                      textcoords="offset points", xytext=(0,10),
                      ha='center', fontsize=9, fontweight='bold')

# Plot 3: ringSep vs mean innerRingRadii
scatter2_3 = axes2[2].scatter(ring_sep, mean_radai,
                              c=shimmed_mean_mT, s=150,
                              cmap=cmap2, norm=norm2,
                              edgecolors='black', linewidth=1.5, alpha=0.8)
axes2[2].set_xlabel('Ring Separation (mm)', fontsize=12, fontweight='bold')
axes2[2].set_ylabel('Mean Inner Ring Radius (mm)', fontsize=12, fontweight='bold')
axes2[2].set_title('Mean Field vs Separation & Radius', fontsize=13, fontweight='bold')
axes2[2].grid(True, alpha=0.3)

for i, (x, y, m) in enumerate(zip(ring_sep, mean_radai, shimmed_mean_mT)):
    axes2[2].annotate(f'{m:.1f}', (x, y),
                      textcoords="offset points", xytext=(0,10),
                      ha='center', fontsize=9, fontweight='bold')

# Add colorbar for Figure 2
fig2.subplots_adjust(right=0.92)
cbar_ax2 = fig2.add_axes([0.94, 0.15, 0.02, 0.7])
cbar2 = fig2.colorbar(scatter2_3, cax=cbar_ax2)
cbar2.set_label('Field Mean (mT)', rotation=270, labelpad=25,
                fontsize=12, fontweight='bold')

plt.suptitle('Parameter Effects on Mean Field Strength',
             fontsize=16, fontweight='bold', y=0.98)
plt.tight_layout(rect=[0, 0, 0.92, 0.96])

return fig1, fig2
print("Running main parameter sweep...")
results_list, field_data_list = load_results('main_params_sweep')

if results_list is None:

```

```

print("No saved results found. Running simulations...")
results_list, field_data_list = run_parameter_sweep(
    halbach_array_simulation,
    different_param_passes,
    show_progress=True,
    save_fields=True
)
save_results(results_list, field_data_list, 'main_params_sweep')
else:
    print("Using previously-saved results.")
# Create pair-plots
fig_params = plot_parameter_pairs(results_list)
plt.show()
fig = plot_field_metrics_histogram(results_list)
plt.show()
optimal_result = halbach_array_simulation(8, np.array([140, 145, 150, 155]) * 1e-3, np.array([48, 50, 52, 54]), 25, True)
def plot_field_strength_vs_homogeneity(all_results):
    """
    Plot mean magnetic field strength versus homogeneity for each configuration.

    Each simulation appears as a single point. This visualization highlights
    parameter combinations that maximize field strength while minimizing homogeneity.

    all_results: Simulation result entries containing `field_details`.

    outputs:
    fig: Scatter plot showing strength-homogeneity trade-off.
    """
    # Extract data - handle both formats
    field_strength = []
    homogeneity_ppm = []
    names = []

    for r in all_results:
        # Check if it's from run_parameter_sweep
        if 'field_details' in r:
            field_strength.append(r['field_details']['Shimmed mean'])
            homogeneity_ppm.append(r['field_details']['Shimmed homogeneity (ppm)'])
            names.append(r.get('config_name', r.get('name', 'Unknown')))
        # Or from run_simulation (one simulation's format for output isn't a dictionary, just one
list)

    else:
        field_strength.append(r['mean_field_mT'])
        homogeneity_ppm.append(r['homogeneity_ppm'])
        names.append(r['name'])

    # Create figure
    fig, ax = plt.subplots(figsize=(12, 8))

    # Find best configuration (lowest homogeneity)
    best_idx = homogeneity_ppm.index(min(homogeneity_ppm))

    # Create scatter plot with different colors
    colors = ['green' if i == best_idx else 'steelblue' for i in range(len(names))]
    scatter = ax.scatter(field_strength, homogeneity_ppm,
                        s=200, c=colors, alpha=0.6,
                        edgecolors='black', linewidth=2)

    # Add labels for each point
    for i, (x, y, name) in enumerate(zip(field_strength, homogeneity_ppm, names)):
        # Offset the text slightly to avoid overlapping with points
        offset_x = 10
        offset_y = 10 + (i % 5) * 8

```

```

# Adjust offset for best configuration
if i == best_idx:
    ax.annotate(name, (x, y),
                xytext=(0.05, 25),
                textcoords='offset points',
                fontsize=16, fontweight='bold',
                bbox=dict(boxstyle='round,pad=0.5',
                        facecolor='lightgreen',
                        edgecolor='green',
                        linewidth=2, alpha=0.8),
                arrowprops=dict(arrowstyle='->',
                                connectionstyle='arc3,rad=0',
                                color='green', linewidth=2))
else:
    ax.annotate(name, (x, y),
                xytext=(offset_x, offset_y),
                textcoords='offset points',
                fontsize=16,
                bbox=dict(boxstyle='round,pad=0.3',
                        facecolor='white',
                        alpha=0.7),
                arrowprops=dict(arrowstyle='->',
                                connectionstyle='arc3,rad=0.2',
                                alpha=0.5))

# Labels and formatting
ax.set_xlabel('Field Strength (mT)', fontsize=18, fontweight='bold')
ax.set_ylabel('Homogeneity (ppm)', fontsize=18, fontweight='bold')
ax.set_title('Field Strength vs Homogeneity Trade-off',
             fontsize=20, fontweight='bold', pad=20)
ax.grid(True, alpha=0.3, linestyle='--')

# Add reference lines for the best configuration
ax.axhline(y=min(homogeneity_ppm), color='green',
           linestyle='--', alpha=0.3, linewidth=2,
           label=f'Best Homogeneity: {min(homogeneity_ppm):.0f} ppm')
ax.axvline(x=field_strength[best_idx], color='green',
           linestyle='--', alpha=0.3, linewidth=2,
           label=f'Best Field: {field_strength[best_idx]:.2f} mT')

# Add legend
ax.legend(loc='upper right', fontsize=16, framealpha=0.9)

plt.tight_layout()
return fig
fig1 = plot_field_strength_vs_homogeneity(results_list)
plt.show()

```